

```

Case 16: Word = "Sixteen"
Case 17: Word = "Seventeen"
Case 18: Word = "Eighteen"
Case 19: Word = "Nineteen"
Case 20: Word = "Twenty"
Case 30: Word = "Thirty"
Case 40: Word = "Forty"
Case 50: Word = "Fifty"
Case 60: Word = "Sixty"
Case 70: Word = "Seventy"
Case 80: Word = "Eighty"
Case 90: Word = "Ninety"
Case 100: Word = "Hundred"
Case 1000: Word = "Thousand"
Case 100000: Word = "Lakh"
' comment above one line and
uncomment below one line if currency
is in dollars
' Case 100000: Word = "Hundred"
Case 1000000: Word = "Million"
' Useful of US/UK naming system
Case 100000000: Word = "Crore"
Case 1000000000: Word = "Billion"
' Useful of US/UK naming system
End Select
End Function

```

The second function, TwoDigitToWord, returns the word equivalents of two-digit decimal numbers including numbers not defined (e.g., 25, 34, etc) in the Word function described above.

A sample code using TwoDigitToWord function is listed below:

```

Function TwoDigitToWord(ByVal
intNum2 As Integer) As String
Dim intA1 As Integer, intA2 As
Integer
Dim strResult3 As String, intDiv
As Integer

intA1 = intNum2
strResult3 = ""

If intA1 > 19 Then
Do While (intA1 > 0)
intDiv = CInt("1" &
String(Len(CStr(intA1)) - 1, "0"))
intA2 = Int(intA1 / intDiv) *
intDiv

strResult3 = strResult3 &
Space(1) & Word(intA2)
intA1 = intA1 Mod intA2

```

```

Loop
Else
strResult3 = Word(intA1)
End If

TwoDigitToWord = LTrim(strResult3)
End Function

```

First, it checks whether the argument passed is greater than 19. If not, it simply calls the Word function. If yes, it extracts the digit from ten's place of the number, appends 0 to it, finds the word equivalent of this number by calling the Word function, then again finds the word equivalent of the digit in unit's place if it's not zero. Finally, these two word equivalents are combined to give the word equivalent of the argument passed.

For example, if the argument passed is 37, the program first finds the word equivalent of 30 (thirty) and then the word equivalent of 7 (seven). Now, it combines these words to give the final result (thirty seven).

The third function, ThreeDigitToWord, gives the word equivalents of three-digit numbers.

For example, Rs 9178434685.50 is grouped as 917,84,34,685.50. This number is read as 'rupees nine hundred seventeen crore eighty four lakh thirty four thousand six hundred eighty five and paise fifty only' in Indian currency and number. Its conversion is explained in the fourth function. The following code is used to convert three-digit numbers into word equivalents:

```

Function ThreeDigitToWord(ByVal
intNum1 As Integer) As String

Dim strY1 As String, strY2 As String
Dim strResult2 As String, intLen As
Integer

strY1 = intNum1

'find the length of the string
intLen = Len(strY1)
strResult2 = ""

```

```

Do While (intLen > 0)
intLen = Len(strY1)

Select Case intLen

Case 3:
strY2 = Mid(strY1, 1, 1)
strResult2 = strResult2 & " " &
TwoDigitToWord(strY2)
strResult2 = strResult2 & " " &
Word(intLen)
strY1 = Mid(strY1, 2, intLen)

Case 2, 1:
strResult2 = strResult2 & " " &
TwoDigitToWord(strY1)
strY1 = "0"

End Select
strY1 = Int(strY1)
If Int(strY1) = 0 Then intLen = 0

Loop

ThreeDigitToWord = LTrim(strResult2)
End Function

```

The fourth function, toWords, gives the word equivalent of a number, say, Rs 9178434685.50, using the following logic:

```

Function toWords(ByVal varNum As
Variant) As String
Dim intLen As Integer, strX1 As
String
Dim strX2 As String, strResult1 As
String
Dim intPos As Integer
' round the number to two decimal
places
varNum = Round(varNum, 2)
strX1 = CStr(Int(varNum))
'find the length of the string
intLen = Len(strX1)
strResult1 = ""
Do While (intLen > 0)
intLen = Len(strX1)
Select Case intLen

Case 10:
strX2 = Mid(strX1, 1, 3)
strResult1 = strResult1 & " " &
ThreeDigitToWord(strX2)

```